

LAB 9.1

Scenario

In this lab you will perform advanced hacking techniques hunting at the endpoint level. Specifically, you will learn how to hunt for:

- **AMSI bypasses**
- **Parent PID spoofing** and
- **Reflective DLL injection**
- **.Net malware**

Goals

The learning objective of this lab is to increase familiarity with some commonly used and advanced hacking techniques.

What you will learn

You will learn how to utilize available alternative data sources and uncommon tools to detect the abovementioned attacks.

Recommended tools

- AmsiPatchDetection
- Custom Scripts
- memhunter
- Process Hacker 2

All the tools are present in **C:\Users\Administrator\Desktop\Tools** folder.

Tasks

Task 1. Hunt for AMSI bypasses

The Windows Antimalware Scan Interface (AMSI) is a versatile interface standard that allows your applications and services to integrate with any antimalware product that's present on a machine. While applications and services are running they can ask AMSI, "Is this line of code I'm about to run malicious?" AMSI is also an effective tool against certain obfuscation and evasion techniques, as it is queried immediately before each line is run, after any deobfuscation performed by the attacker.

This functionality is provided by *amsi.dll*. Thus, we can tell if a program potentially uses AMSI by the presence of this DLL in its loaded modules.

Let's take for example PowerShell. Any process has full permissions to modify its own internal memory space. As *amsi.dll* is loaded within the memory space of the PowerShell process, an attacker can use their PowerShell commands to attack AMSI itself.

Attackers start by finding the in memory location of the AMSI function, they introduce code that modifies the memory protections to be read, write, execute (you can verify this with Process Hacker 2) and finally, they re-write the AMSI function so that it always returns a negative ("this is not malware"). In other words, they are patching *amsi.dll* in memory.

Execute the provided AMSI bypass code and try to hunt for the changes it makes.

The AMSI bypass code (*ASBBypass.ps1*) is located in the *Downloads* directory.

Hint: The answer lies deep within Windows' memory management mechanics. It is not uncommon to see two (or more) processes using the same DLL (a good example is ntdll.dll). For optimization purposes, Windows doesn't load such a DLL multiple times. Windows load it once and each process refers to the memory location where the DLL was loaded. Does this mean that if code (DLL) is edited at runtime, all processes that refer to the same shared memory location will be affected as well? Luckily no! Before Image memory (DLL, EXE, etc.) is edited, Windows takes a copy of this chunk of memory, and makes it private to the process doing the editing. This is known as "Copy on Write" memory.

The real hint here is that Windows provides some system calls, allowing us to ask, for a given memory address in a given process, "Is this memory shared?" - or effectively, "Is it copy-on-write?" If we find the location of the *amsi.dll* in memory, and find that it is non-shared memory, this is a strong indication that this memory has been modified in the past, indicating that an AMSI bypass has been run.

A tool that performs the check above can be found below.

Task 2. Hunt for Parent PID Spoofing

Blue team members heavily utilize anomalies parent-child process relationships in order to uncover evil in their environments. Advanced adversaries can bypass this using Parent PID (PPID) Spoofing that allows for the execution of a malicious process from an arbitrary parent process.

For more information refer to the links below:

- <https://blog.didierstevens.com/2017/03/20/>
- <https://blog.didierstevens.com/2009/11/22/quickpost-selectmyparent-or-playing-with-the-windows-process-tree/>
- <https://blog.cobaltstrike.com/2017/05/23/cobalt-strike-3-8-whos-your-daddy/>

There are multiple ways to spoof a parent of a process. We will focus on the most commonly used one that involves the API call [*CreateProcessA*](#). This function features a parameter called "*lpStartupInfo*" where one can specify the parent process he/she wants to use. To be more technically accurate, the *lpStartupInfo* parameter is a pointer to a [*STARTUPINFOEX*](#) structure. This structure contains an attribute list (*lpAttributeList*) and using [*UpdateProcThreadAttribute*](#) you can set the process parent via the "*PROC_THREAD_ATTRIBUTE_PARENT_PROCESS*" attribute.

One of the best forensic data sources in Windows is Event Tracing for Windows (ETW). ETW provides a real-time stream of data about events occurring on a system. Utilize ETW in order to hunt for PPID Spoofing.

You can emulate PPID Spoofing using a PowerShell script (*PPID-Spoof.ps1*) that resides at *C:\Users\Administrator\Desktop\Tools\ppid-spoofing*

Hint: For our detection purposes, the *Microsoft-Windows-Kernel-Process* provider can give us valuable insight into process creation. Specifically, the *ProcessID* value of the event header should match the *ParentProcessID* value. Be aware that legitimate spoofing may take place.

A script that can help you hunt for PPID Spoofing can be found at the *C:\Users\Administrator\Desktop\Tools\ppid-spoofing* directory.

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Hunt for AMSI bypasses

To hunt for the AMSI bypass using the provided code (*ASBBypass.ps1*), execute the following.

- Execute *cmd.exe* **as an administrator**
- Execute the AMSI bypass detection executable located at *C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug*

You should see something similar to the below.

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.17763.1457]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>cd C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug
C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug>AmsiPatchDetection.exe
Running AMSI patch detection (64-bit only)
C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug>_
```

- Execute powershell.exe and execute the detection executable once again. You should see something similar to the below (PowerShell uses AMSI).

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.17763.1457]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>cd C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug
C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug>AmsiPatchDetection.exe
Running AMSI patch detection (64-bit only)
C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug>AmsiPatchDetection.exe
Running AMSI patch detection (64-bit only)
powershell (3908) uses AMSI
C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug>

Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator>
```

- Emulate the AMSI bypass code (*ASBBypass.ps1*) located in the *Downloads* directory, by executing the below inside the PowerShell terminal you previously spawned.

```
powershell -ep bypass
cd C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection
.\ASBBypass.ps1
```

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> powershell -ep bypass
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> cd C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection
PS C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection> .\ASBBypass.ps1
True
PS C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection> █
```

- Finally, execute the AMSI bypass detection executable again. You should now see it detecting the AMSI bypass on the PowerShell process.

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.17763.1457]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>cd C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug
C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug>AmsiPatchDetection.exe
Running AMSI patch detection (64-bit only)
C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug>AmsiPatchDetection.exe
Running AMSI patch detection (64-bit only)
powershell (3908) uses AMSI
C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug>AmsiPatchDetection.exe
Running AMSI patch detection (64-bit only)
powershell (2688) uses AMSI
*** Potential AMSI bypass detected: powershell (2688) ***
powershell (1228) uses AMSI
C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection\AmsiPatchDetection\bin\Debug>

Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> powershell -ep bypass
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> cd C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection
PS C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection> .\ASBBypass.ps1
True
PS C:\Users\Administrator\Desktop\Tools\AmsiPatchDetection> █
```

If you start *Process Hacker 2* as an administrator, you will also be able to see the memory location of *amsi.dll* having read, write and execute rights (this is not a default behavior, it was caused by the AMSI bypass code).

Recycle Bin

Administrator: Command Prompt

Microsoft Windows [Version 10.0.17763.14571]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>

Process Hacker [ATTACKDEFENSE\Administrator]+

Hacker View Tools Users Help

Refresh Options Find handles or DLLs System information power

Processes Services Network Disk

Name	PID	CPU	I/O total ...	Private b...	User name	Description
svchost.exe	920			6.56 MB	NT AUTHORITY\SYSTEM	Host Process for Windows Ser...
powershell.exe	1228			59.49 MB	ATTAC...\Administrator	Windows PowerShell
powershell.exe	2688	0.01		66.52 MB	ATTAC...\Administrator	Windows PowerShell

powershell.exe (2688) Properties

Handles .NET assemblies .NET performance GPU Disk and Network Comment

General Statistics Performance Threads Token Modules Memory Environment

Name	Base address	Size	Description
powershell.exe	0x7ff6e6f20000	448 kB	Windows PowerShell
advapi32.dll	0x7fffe6f00000	652 kB	Advanced Windows 32 Base API
amsi.dll	0x7fffd9ae0000	80 kB	Anti-Malware Scan Interface
atl.dll	0x7fffd7700000	112 kB	ATL Module for Windows XP (Unicode)
bcrypt.dll	0x7fffe5200000	152 kB	Windows Cryptographic Primitives Library
bcryptprimitives...	0x7fffe6080000	504 kB	Windows Cryptographic Primitives Library
cfgmgr32.dll	0x7fffe5a90000	296 kB	Configuration Manager DLL
clbcatq.dll	0x7fffe9070000	648 kB	COM+ Configuration Catalog
clr.dll	0x7fffcff30000	9.93 MB	Microsoft .NET Runtime Common Language Runtime - WorkS
clrjit.dll	0x7fffcf1b0000	1.17 MB	Microsoft .NET Runtime Just-In-Time Compiler
combase.dll	0x7fffe8d40000	3.18 MB	Microsoft COM for Windows
coml2.dll	0x7fffe8cc0000	468 kB	Microsoft COM for Windows
crypt32.dll	0x7fffe5e80000	1.86 MB	Crypto API32
crypt32.dll.mui	0x2450000	40 kB	Crypto API32
cryptbase.dll	0x7fffe4b00000	48 kB	Base cryptographic API DLL
cryptsp.dll	0x7fffe6350000	92 kB	Cryptographic Service Provider API
gdi32.dll	0x7fffe6370000	164 kB	GDI Client DLL
gdi32full.dll	0x7fffe61b0000	1.61 MB	GDI Client DLL
gpapi.dll	0x7fffe3db0000	136 kB	Group Policy Client API
iertutil.dll	0x7fffd6900000	2.66 MB	Run time utility for Internet Explorer
imm32.dll	0x7fffe8a50000	184 kB	Multi-User Windows IMM32 API Client DLL
kernel.appcore.dll	0x7fffe5160000	68 kB	AppModel API Host
kernel32.dll	0x7fffe8820000	716 kB	Windows NT BASE API Client DLL
KernelBase.dll			

CPU Usage: 0.98% Physical memory:

nts reserved.

pass

nts reserved.

istrator\Desktop\Tools\AmsiPatchDetection

PatchDetection> .\ASBypass.ps1

PatchDetection>

Task 2. Hunt for Parent PID Spoofing

To hunt for PPID Spoofing using the provided code (*PPID-Spoof.ps1*), execute the following.

- Execute *cmd.exe* **as an administrator**
- Execute the PPID Spoofing detection script (*detect-ppid-spoof.py*) located at *C:\Users\Administrator\Desktop\Tools\ppid-spoofing*

You should see something similar to the below.

```
Administrator: Command Prompt - python detect-ppid-spoof.py
Microsoft Windows [Version 10.0.17763.1457]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>cd C:\Users\Administrator\Desktop\Tools\ppid-spoofing
C:\Users\Administrator\Desktop\Tools\ppid-spoofing>python detect-ppid-spoof.py
_
```

- Emulate the attack as follows.
 - Use the task manager and note *explorer.exe*'s PID)
 - Execute *powershell.exe* **as an administrator** and then, execute the below

```
powershell -ep bypass
cd C:\Users\Administrator\Desktop\Tools\ppid-spoofing
import-module .\PPID-Spoof.ps1
PPID-Spoof -ppid 4752 -spawnTo "C:\Windows\System32\notepad.exe" -dllpath C:\Users\Administrator\Desktop\Tools\memhunter\msimplepayload.dll
```

You should see the below.

Process Hacker [ATTACKDEFENSE\Administrator]+

Hacker View Tools Users Help

Refresh Options Find handles or DLLs System information explorer

Name	PID	CPU	I/O total ...	Private b...	User name	Description
explorer.exe	4752	0.48		29.36 MB	ATTAC...Administrator	Windows Explorer

Administrator: Windows PowerShell

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

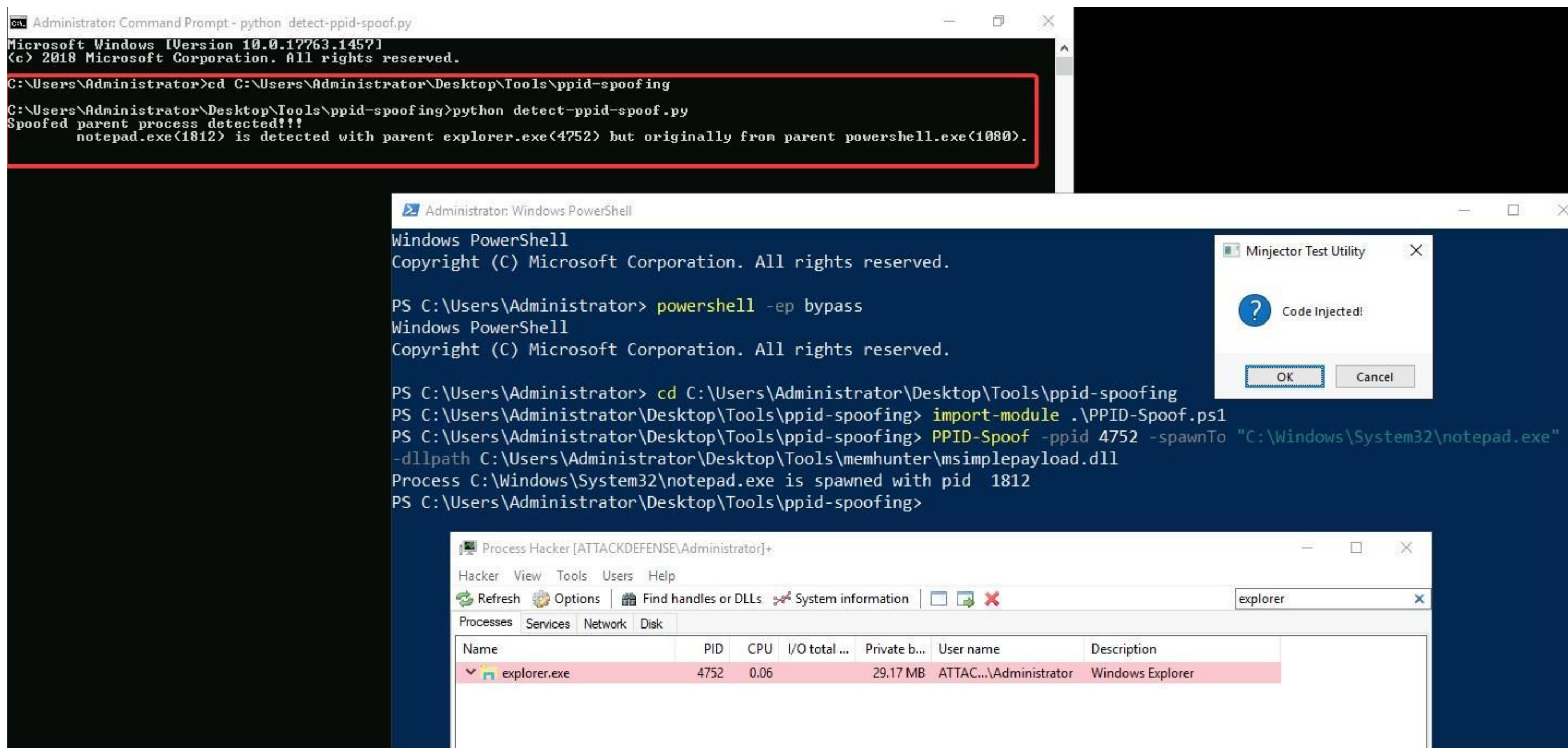
PS C:\Users\Administrator> powershell -ep bypass
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> cd C:\Users\Administrator\Desktop\Tools\ppid-spoofing
PS C:\Users\Administrator\Desktop\Tools\ppid-spoofing> import-module .\PPID-Spoof.ps1
PS C:\Users\Administrator\Desktop\Tools\ppid-spoofing> PPID-Spoof -ppid 4752 -spawnTo "C:\Windows\System32\notepad.exe"
-dllpath C:\Users\Administrator\Desktop\Tools\memhunter\msimplepayload.dll
Process C:\Windows\System32\notepad.exe is spawned with pid 1812
PS C:\Users\Administrator\Desktop\Tools\ppid-spoofing>
```

Minjector Test Utility

Code Injected!

OK Cancel



The detection script successfully identified the PPID spoofing activity.

References

1. <https://ionize.com.au/detecting-amsi-bypass/>
2. <https://blog.f-secure.com/detecting-parent-pid-spoofing/>